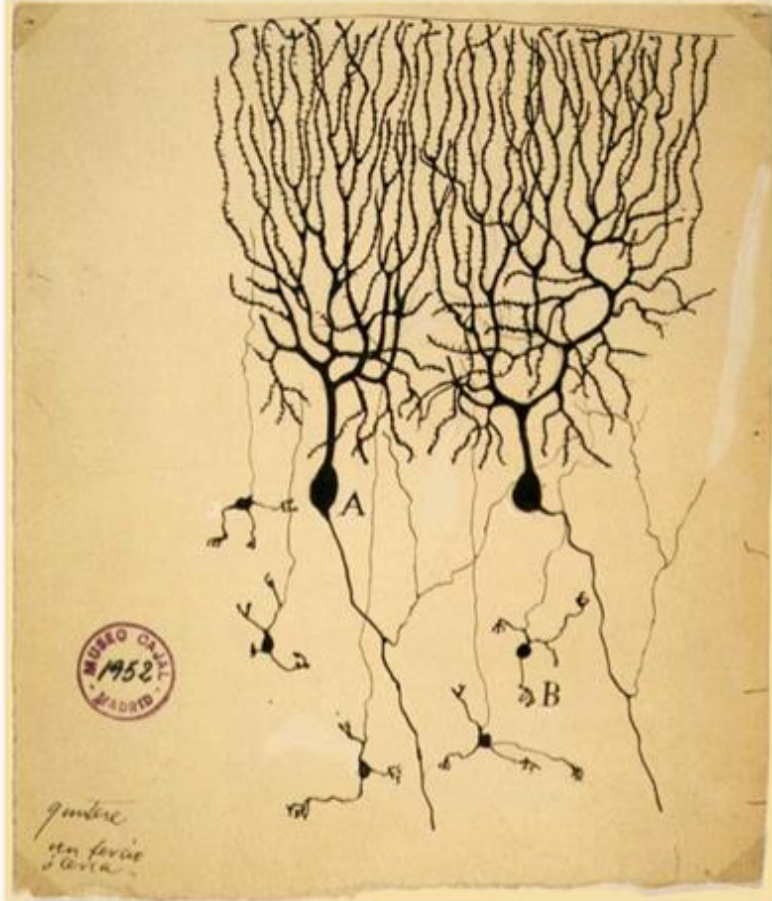


Neural Networks

Forward Pass and Back Propagation

Nimisha Roy
Georgia Tech

Inspiration from Biological Neurons



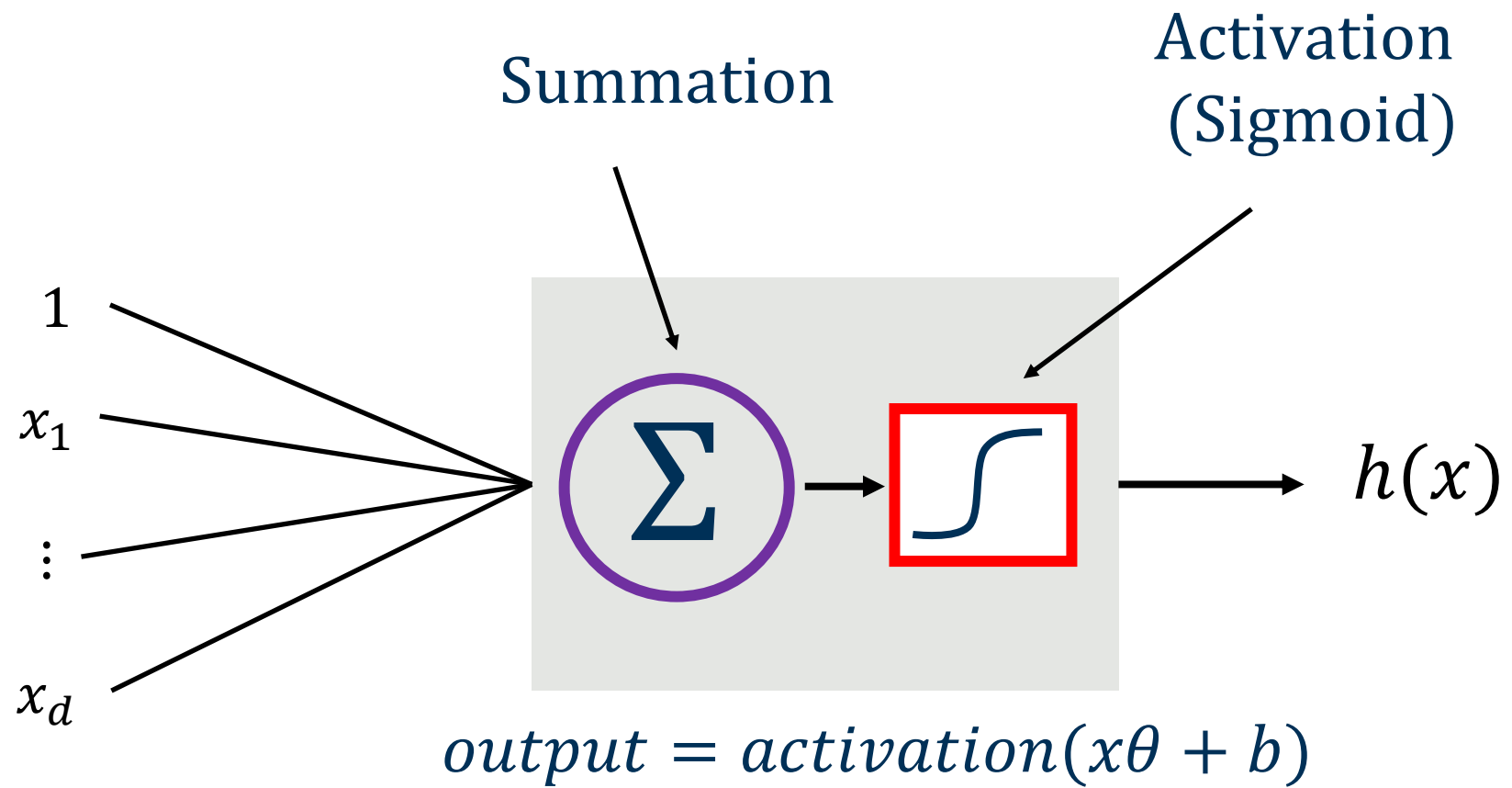
The first drawing of a brain cells by Santiago Ramón y Cajal in 1899

Neurons: core components of brain and the nervous system consisting of

1. Dendrites that collect information from other neurons
2. An axon that generates outgoing spikes

Neural Networks

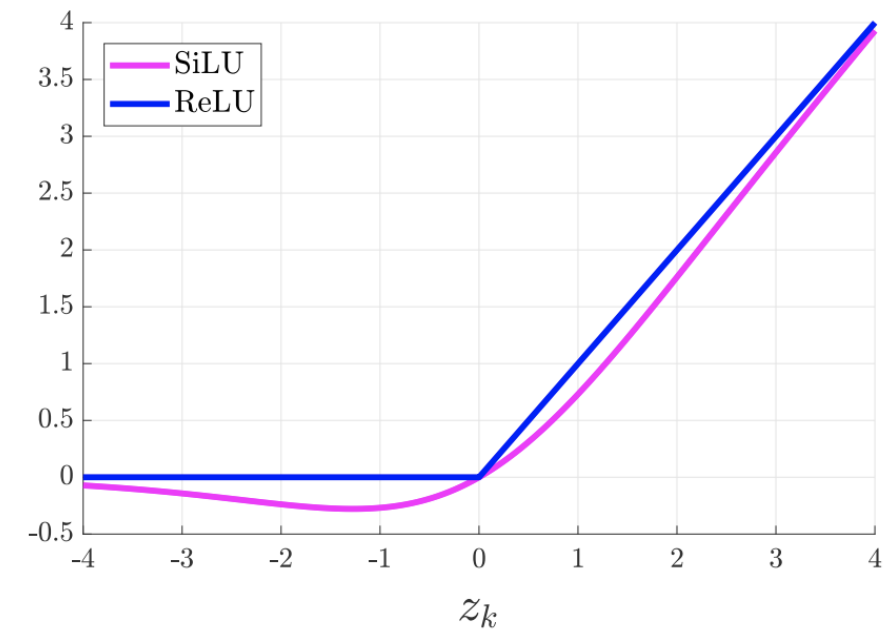
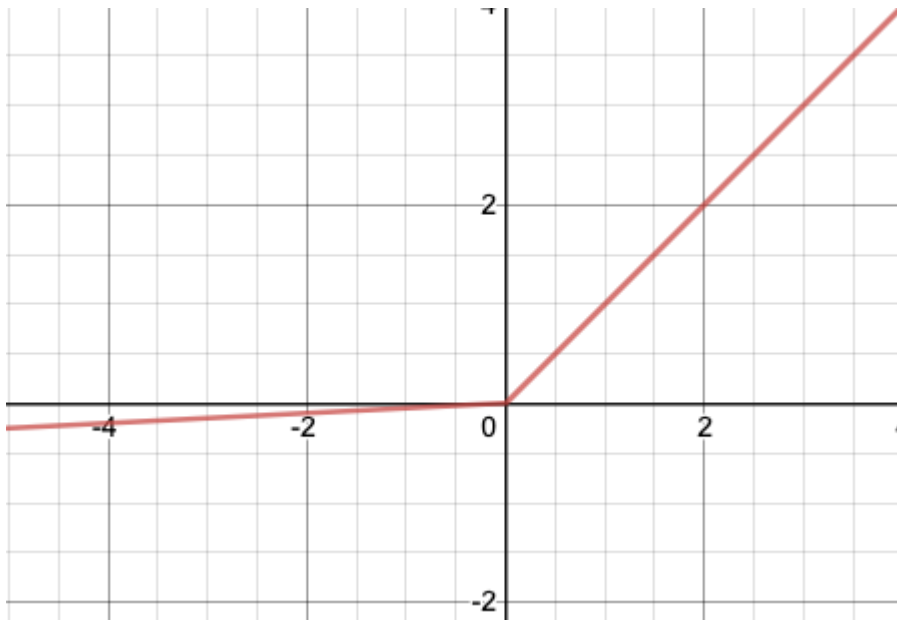
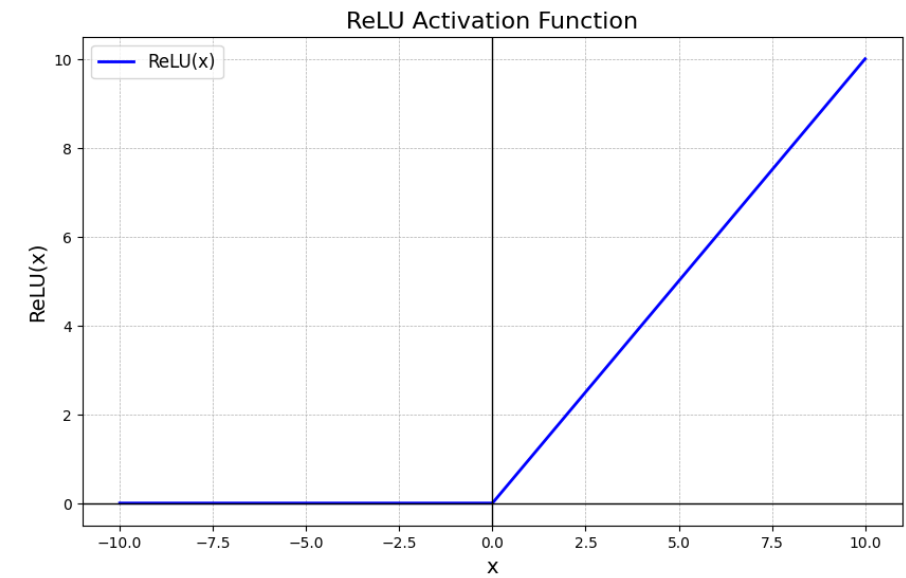
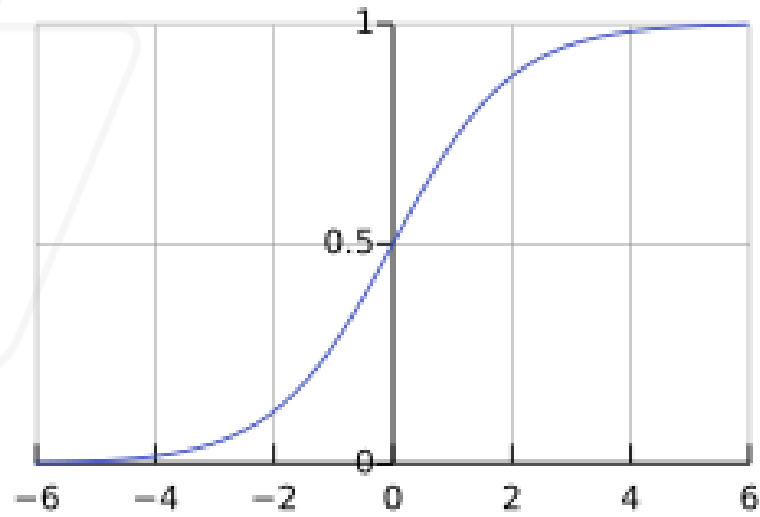
- A robust approach for approximating **real-valued, discrete-valued or vector valued** functions
- Among the most effective **general purpose** supervised learning methods
- Apt at modeling **complex and hard to interpret data** such as images and audio
- The **backpropagation algorithm** for neural networks has been shown successful in many practical problems:
 - Handwritten character recognition, speech recognition, object recognition, some NLP problems



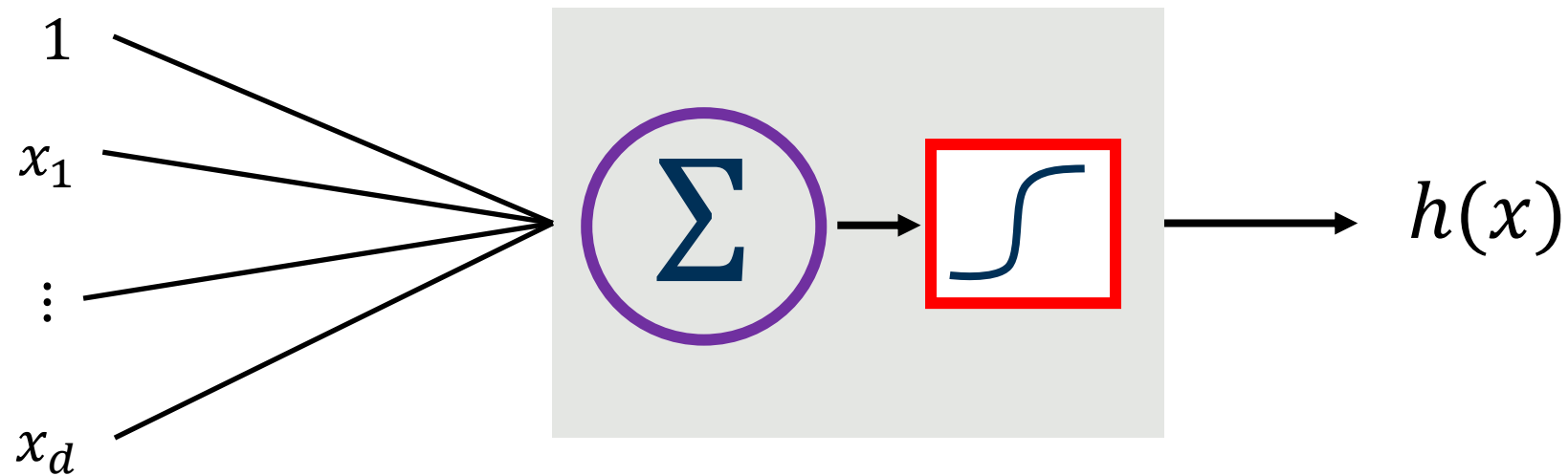
$$h(x) = o(x\theta = u) \rightarrow \text{logistic regression}$$

Name of the neuron	Activation function: $activation(z)$
Linear unit	z
Threshold/sign unit	$sgn(z)$
Sigmoid unit	$\frac{1}{1 + \exp(-z)}$
Rectified linear unit (ReLU)	$\max(0, z)$
Tanh unit	$\tanh(z)$

Different activation functions: o



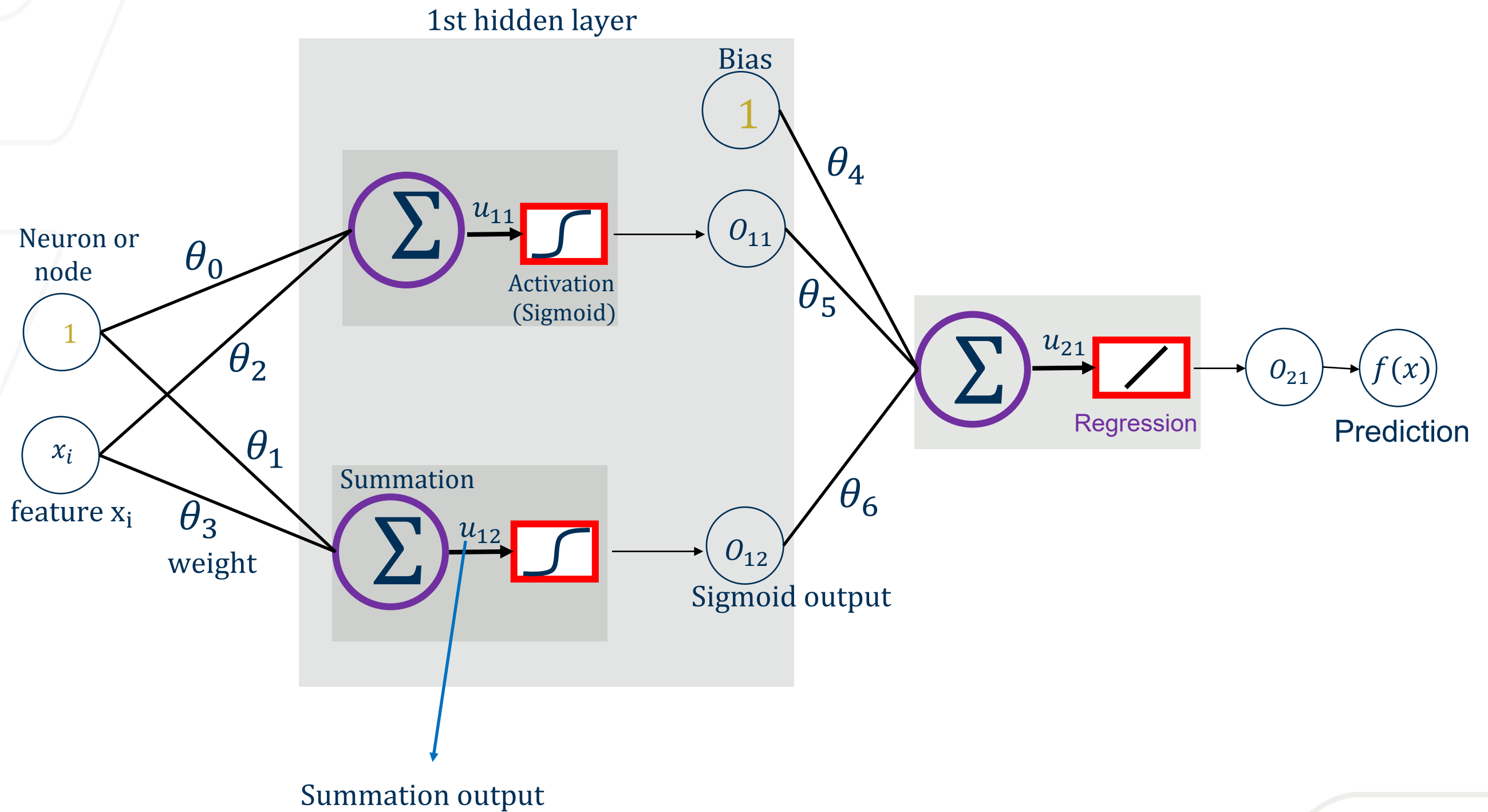
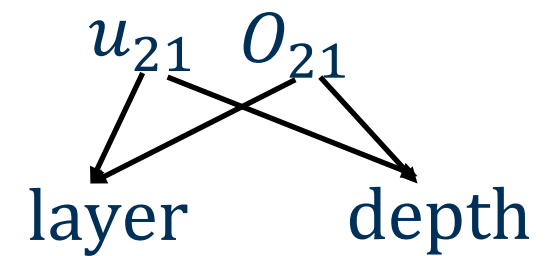
In Neural Network, 1 learning block is 1 building block

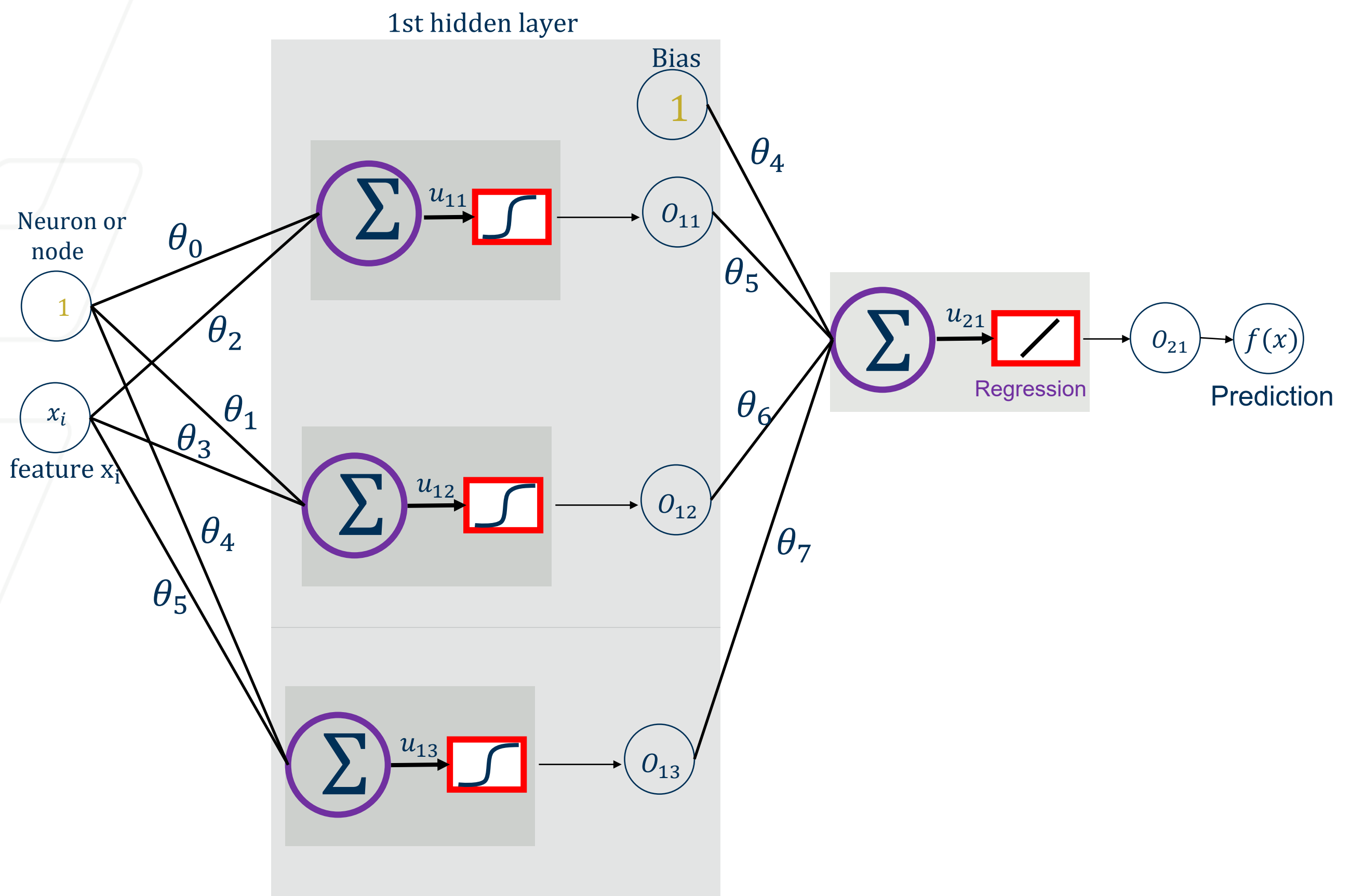


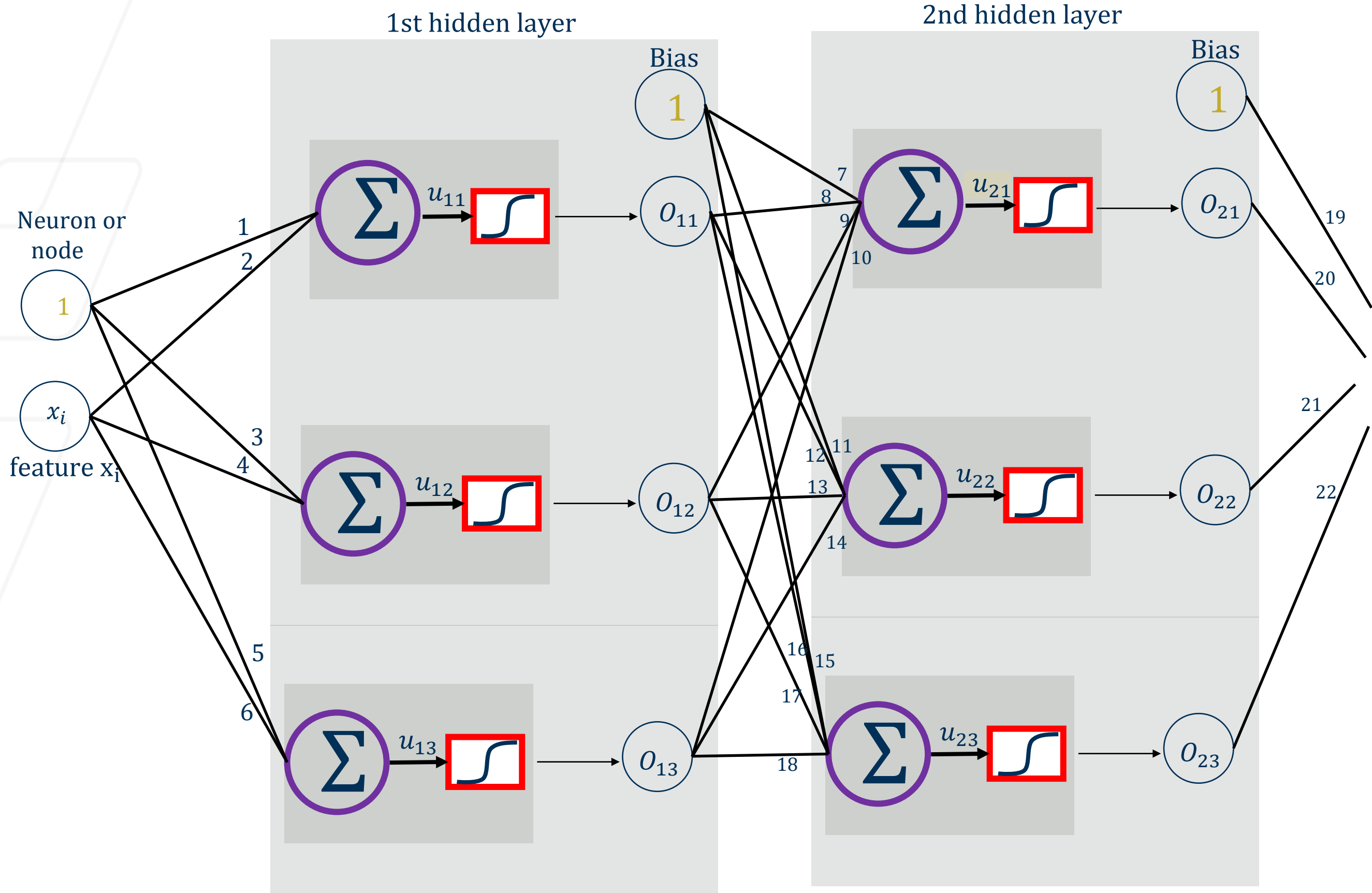
A **neural network** is essentially a **stack (or composition)** of many **learning blocks**, where:

Output of Block i = Input to Block $(i + 1)$

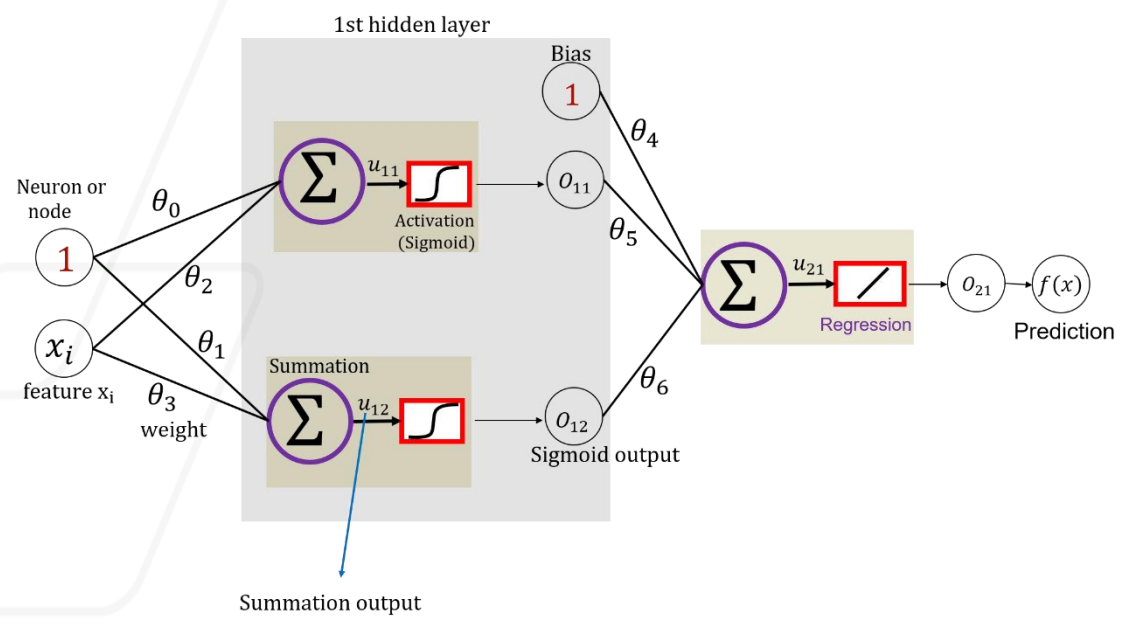
Building a Network: NN Regression



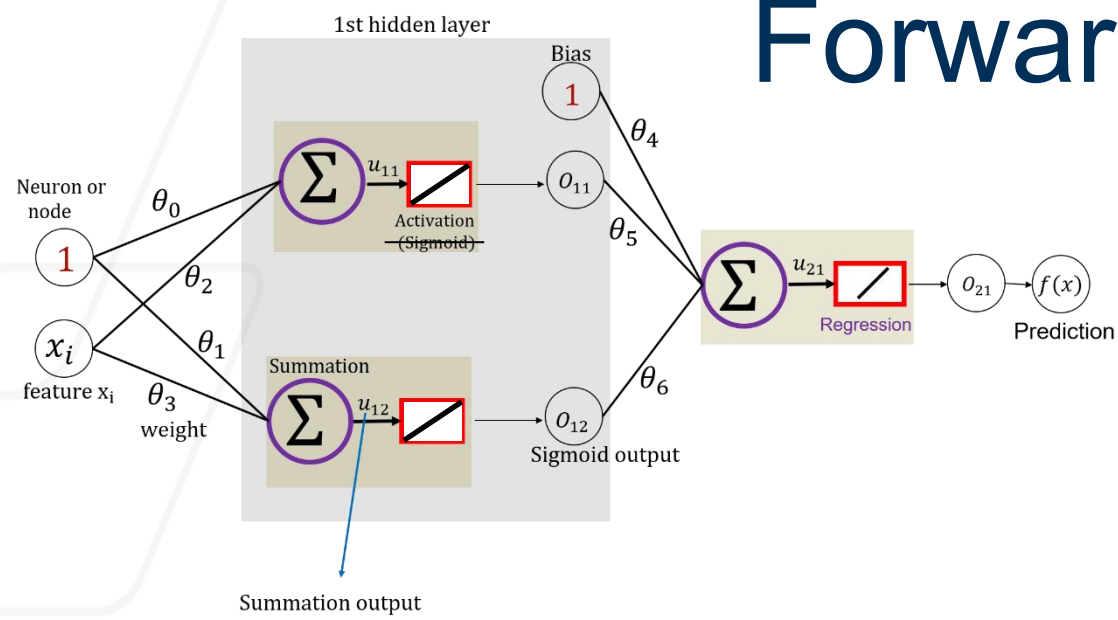




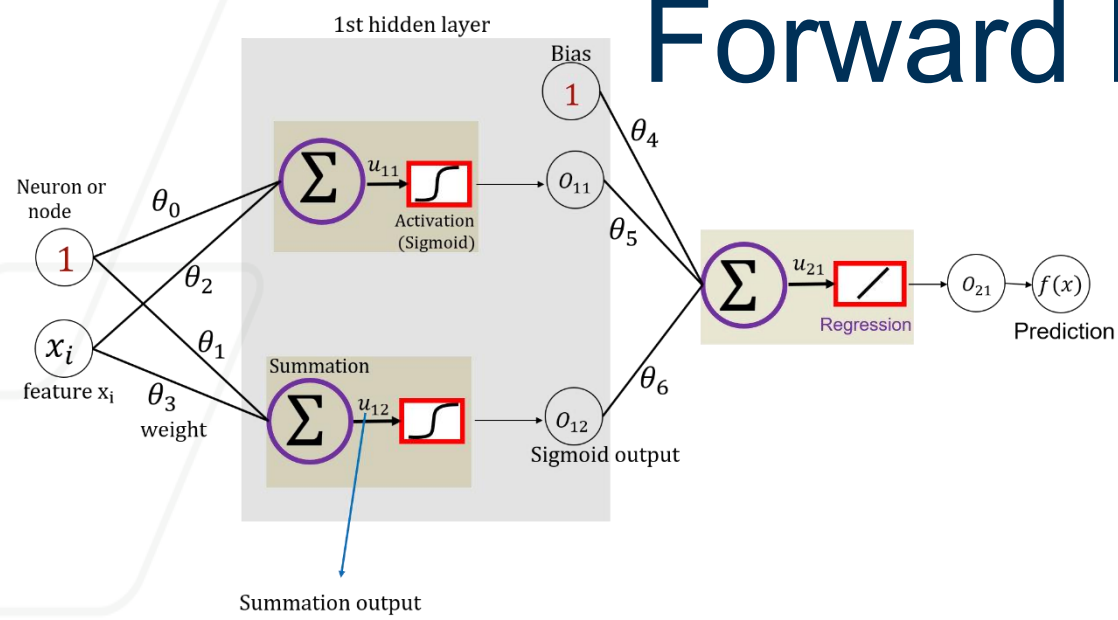
Gradient Descent



Forward Propagation: Rigid Network

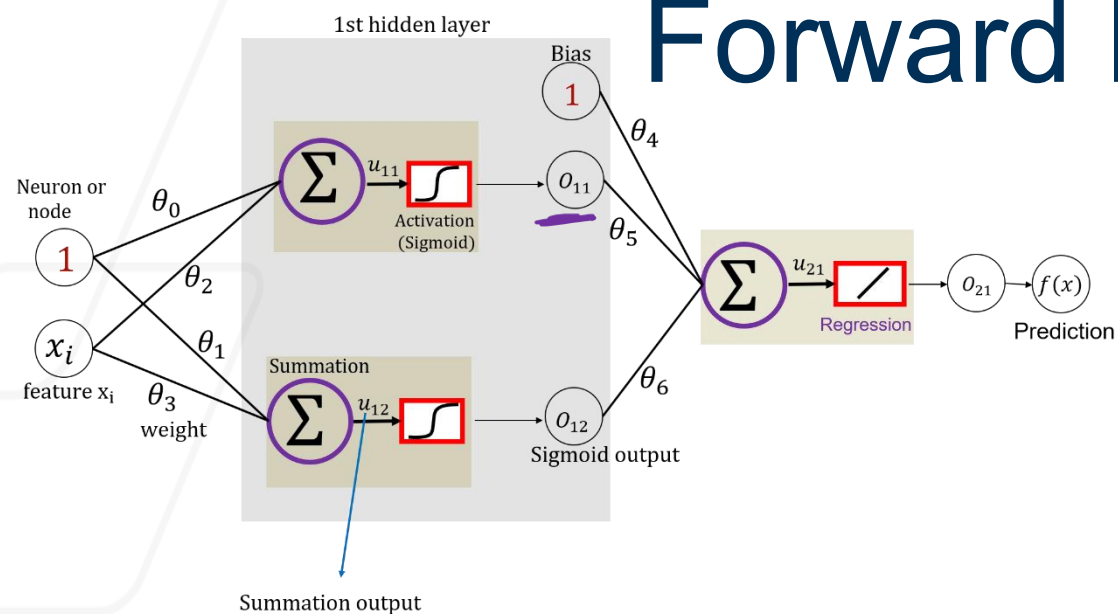


Forward Propagation: Flexible Network



<https://playground.tensorflow.org>

Forward Propagation: Flexible Network



$$u_{11} = \theta_0 + \theta_2 x_i$$

$$o_{11} = \frac{1}{1 + \exp(-u_{11})} = \frac{1}{1 + \exp(-\theta_0 - \theta_2 x_i)}$$



$$u_{12} = \theta_1 + \theta_3 x_i$$

$$o_{12} = \frac{1}{1 + \exp(-\theta_1 - \theta_3 x_i)}$$



$$u_{21} = \theta_4 + \theta_5 o_{11} + \theta_6 o_{12}$$

$$= \theta_4 + \frac{\theta_5}{1 + \exp(-\theta_0 - \theta_2 x_i)} + \frac{\theta_6}{1 + \exp(-\theta_1 - \theta_3 x_i)}$$

$$o_{21} = f(x) = u_{21}$$

vertical translation

vertical shrinking or stretching

horizontal translation

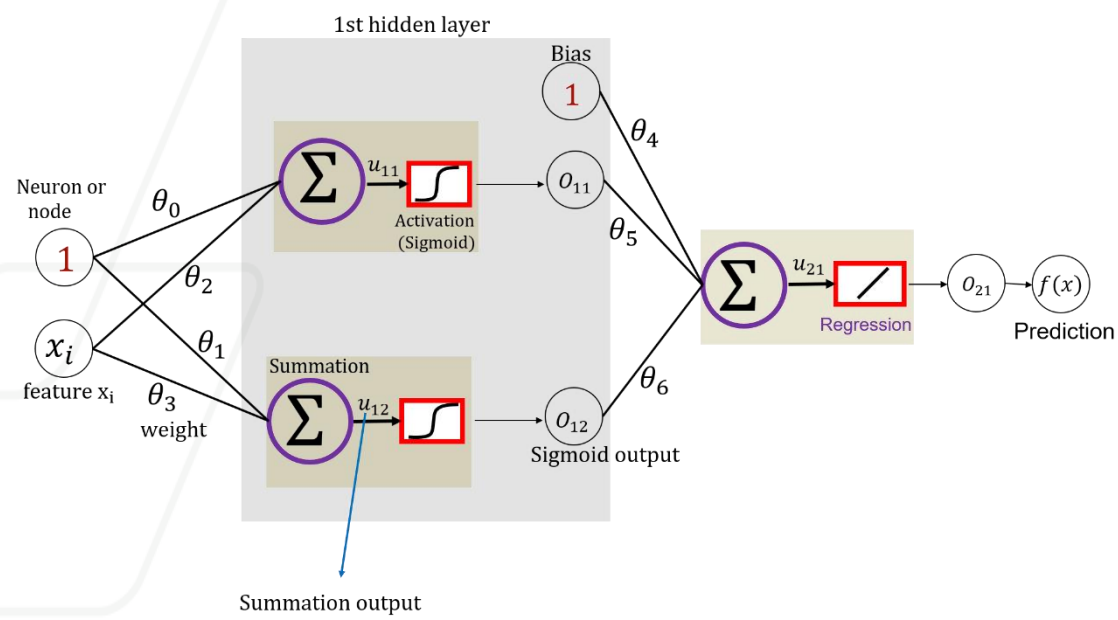
horizontal shrinking or stretching

Recap

- Neural Network – components of a building block
- Classification or Regression?
- activation function – uses and types
- Number of parameters
- GD Step 0 and Step 1

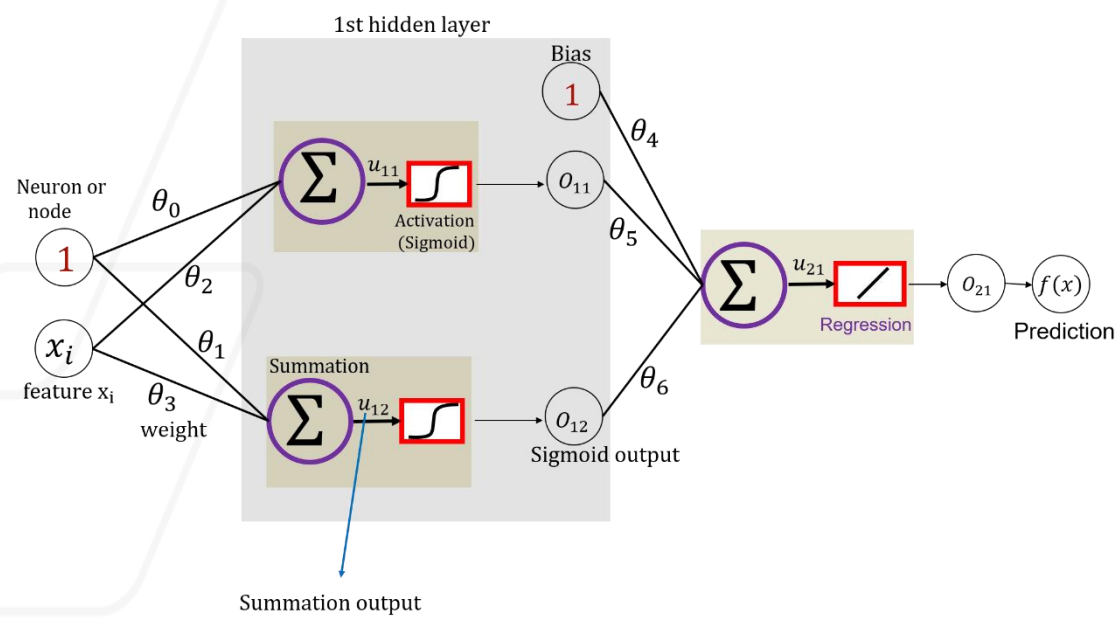
Backward Propagation

Objective Function $E(\theta)$:

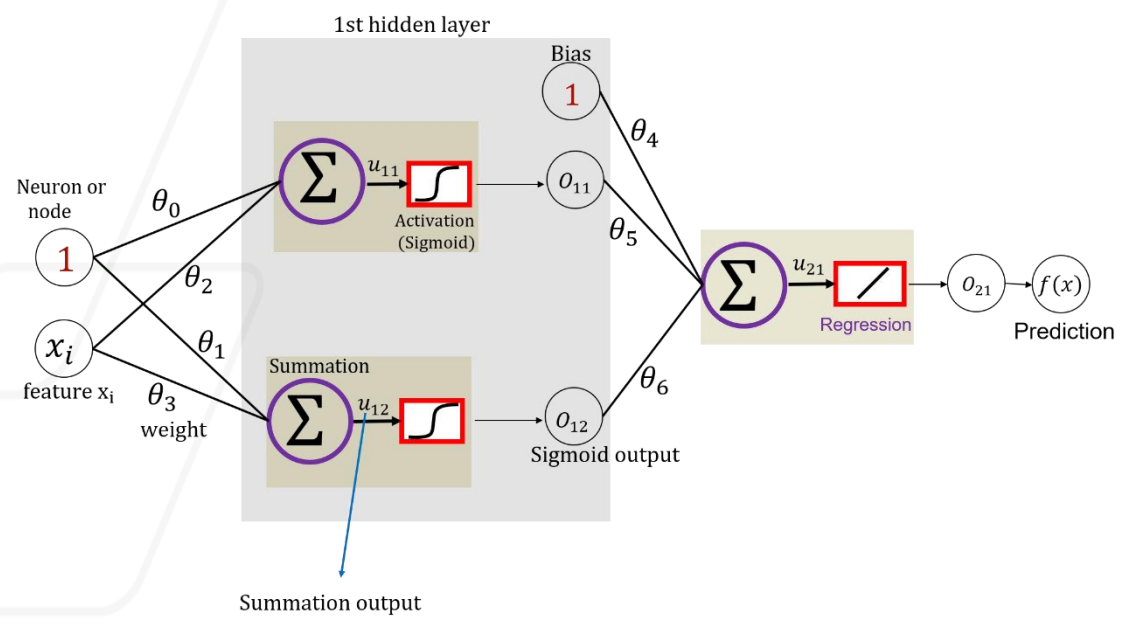


Backward Propagation

Objective Function $E(\theta)$:



Backward Propagation



Vanishing and Exploding Gradient

In deep networks, gradients can **shrink (vanish)** or **grow uncontrollably (explode)** during backpropagation, making training unstable or slow.

Sigmoid Activation

- Output range: $(0, 1)$. Derivative: $\sigma'(x) = \sigma(x)(1 - \sigma(x)) \leq 0.25$
- Large $|x| \rightarrow$ output saturates near 0 or 1 $\rightarrow$ gradients ≈ 0 . $\rightarrow$ **Vanishing gradient problem** $\rightarrow$ early layers stop learning

ReLU Activation

- Derivative: 1 for $x > 0$, 0 for $x \leq 0$
- Avoids vanishing (no upper saturation)
- But:
 - Large positive activations + large weights $\rightarrow$ gradients **amplify** across layers. $\rightarrow$ **Exploding gradients**
 - Many $x \leq 0 \rightarrow$ gradients = 0 $\rightarrow$ **Dead ReLUs**

Mitigation

- **BatchNorm** to stabilize activations - Normalizes layer outputs to have zero mean and unit variance before activation.
- **Gradient clipping** to limit large updates - puts an upper bound on gradient magnitude (e.g., clipping at 1.0).
- Use **ReLU variants** (Leaky ReLU) – avoid dead ReLU.

Direction in Gradient Descent

- Each weight affects the total loss of the network.
- Backpropagation computes the **gradient** (magnitude and direction)— how much the loss changes if a weight changes.
- If the gradient is **positive** → decreasing the weight lowers the loss.
- If the gradient is **negative** → increasing the weight lowers the loss.
- Gradient Descent updates weights by taking small steps opposite to the gradient.
- This helps the model **learn** gradually and minimize error.

Summary: Forward Pass, Backpropagation, and Gradient Descent

① Forward Pass — Compute Outputs

- Inputs move through each layer of the network.
- Each layer applies its weights, bias, and activation function.
- The network produces predictions at the output layer.
- The difference between predictions and true values gives the **loss**.
- We compute all u's and o's.

② Backpropagation — Compute Gradients

- The loss is sent backward through the network.
- Each layer calculates how much it contributed to the total error.
- We compute all gradients. Using the **chain rule**, gradients are propagated from output to input.
- This gives the “direction” each weight should move to reduce error.

③ Gradient Descent — Update Parameters

- The optimizer updates weights slightly in the opposite direction of the gradient.
- The size of the adjustment is controlled by the **learning rate**.
- Repeated passes gradually minimize the loss.
- *Model improves by iteratively reducing error over many epochs.*